CSC3041F
Assignment 1
Solving chess via search

Introduction to Artificial Intelligence

Department of Computer Science
University of Cape Town, South Africa

The rise of chess engines has democratized the game, allowing players to access high-level tactical and positional analysis without a private coach. Leveraging this technological shift, this project implements a chess agent based on the Monte Carlo Tree Search (MCTS) algorithm. Our objective is to evaluate its performance against a random-move baseline, highlighting the effectiveness of the algorithm in complex state spaces.

NOTES

1. Release Date: 18h00, Monday, 2rd of March, 2026
2. There are two (2) question, for a total of 30 marks.
3. You have been provided with a ZIP file named `A1-FilesForStudents.zip`. Inside the archive is an `assets` folder and three Python files: `main.py`, `agent.py`, and `board.py`. You are expected to modify the code in `agent.py`; you may add any necessary helper functions or classes within this same file. To execute the simulation, uncomment line 68 in `main.py` to initiate the game between agents. Do not make any other changes to `main.py`.

Getting started

1. Begin by reviewing the `main.py` file. Please note that this program requires **Python 3** and the **Tkinter** library to function. You can execute the program by running the following command: `python main.py`. Upon execution, a graphical interface will appear, as illustrated in Figure 1.

2. In this simulation, the white pieces are controlled by a baseline agent selecting moves at random, while the black pieces are managed by the Monte Carlo Tree Search (MCTS) agent. The entire solution must be implemented in Python

3. The use of generative AI (GenAI) tools is permitted in this assignment under specific conditions. All code submitted must be your own original work, written entirely by you from scratch. Students are permitted to use GenAI only for brainstorming and debugging. All GenAI use must be explicitly disclosed in submitted work. Failure to follow these guidelines or using GenAI in a manner that violates the Academic Honest Policy or the ComSci GenAI policy will be considered a violation of academic integrity. The listed GenAI tool(s) should be used to enhance understanding and creativity, not replace original thought or fundamental skills such as critical thinking, problem-solving, and writing. Should you have any questions, doubts, or concerns about using GenAI tools in your coursework, please consult your lecturer for clarification and guidance.

4. Your final submission must be a single archive named with your student number (e.g., `STDNUM001.zip`). This ZIP file must contain the `assets` folder and the three core Python files: `main.py`, `agent.py` (containing your original solution), and `board.py`. Consistent with the academic integrity policy, all code must be authored independently. If you utilized Generative AI for non-coding purposes (such as conceptual brainstorming or debugging logic as permitted), you must include a document titled `GenAI.docx`. This file must provide a comprehensive log of the models used, all prompts issued, and the resulting outputs.

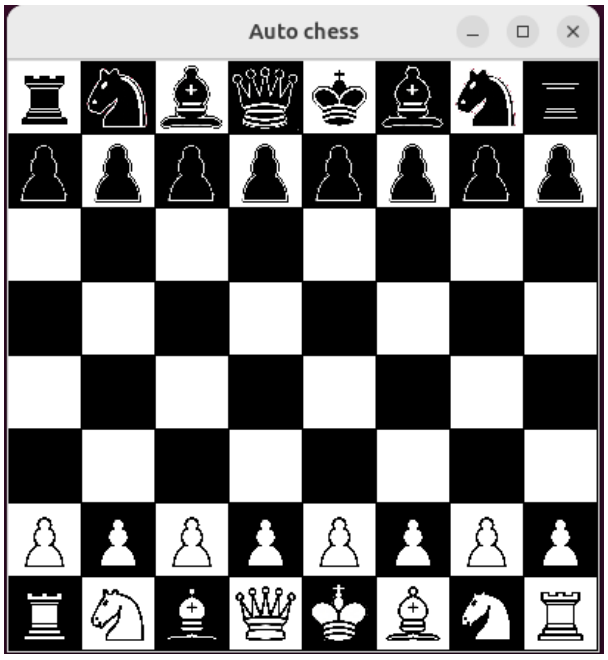


Figure 1: Screenshot of GUI visualising the actions taken by the two agents

1 Playing chess via a search algorithm

The game is played on a standard 8×8 grid, where the white pieces are controlled by a baseline agent selecting actions at random, and the black pieces are managed by the MCTS agent. Piece movement is governed by the *FIDE Laws of Chess* (refer to the provided appendix). Agents alternate turns until a terminal state is reached. For the scope of this assignment, a game limit of **100 plies** (half-moves) is enforced to ensure computational efficiency.

The game board is implemented using a two-tier architecture: `RawChessBoard` and `ChessBoardGUI`. The `RawChessBoard` class serves as the underlying data model, representing the 8×8 grid as a 2D array and providing several essential helper functions for state management. Key methods include:

1. `is_capture`: This method evaluates whether a move constitutes a valid capture. It returns `True` if the piece at the destination coordinates (dest_row, dest_col) is of the opposing color to the piece at the source coordinates (src_row, src_col).

2. `gives_check`: This method determines if executing a move from (src_row, src_col) to (dest_row, dest_col) results in the opponent's king being placed under attack (in check).

3. `get_state_after_move`: This method generates the resulting board state after a player of a specified `color` moves a piece from (curr_x, curr_y) to (next_x,

next_y). It is essential for simulating potential future outcomes without modifying the current game board.

4. `get_playable_moves(color)`: This method identifies all legal moves available to a player of the specified `color`. It returns a list of dictionaries, where each dictionary represents a piece and its potential actions. The output takes the form: {'piece': symbol, 'color': color, 'moves': list_of_destinations, 'row': src_row, 'col': src_col}.

    - `'piece'`: The piece symbol (e.g., 'P', 'N').
    - `'color'`: The player's color.
    - `'moves'`: A list of valid destination coordinates (e.g., [(0,1), (3,3)])
    - `'row' / 'col'`: The current source coordinates of the piece.

The `ChessBoardGUI` class manages the visual representation of the game, rendering the state of the `RawChessBoard` using the `Tkinter` library. It acts as the interface layer, translating the underlying data model into a graphical display for the user.

2 Question 1 - Random agent [10 marks]

1. Complete the `RandomChessAgent` class by implementing the `get_next_move` method. This method must select a valid move uniformly at random from the current board state. The function must return the source and destination coordinates in the following order: `src_row, src_col, dest_row, dest_col`. Ensure that the selected move is strictly among the legal moves provided by the `RawChessBoard` helper functions.

3 Question 2 - Monte Carlo Tree Search agent [20 marks]

1. Implement the `MonteCarloChessAgent` class by completing the `get_next_move` method. The function must also return the source and destination coordinates in the following order: `src_row, src_col, dest_row, dest_col`. Your agent must utilize the MCTS algorithm to select the optimal move.

   For node selection, you must implement the Upper Confidence Bound (UCB1) formula with $C = 1.14$ (see page 163, notes).

   During the rollout, instead of purely random play for the black pieces, you must implement the following ordering function: captures first, check-giving moves, then random moves. The white pieces can select moves randomly. A game limit of 100 plies (half-moves) must be enforced as well. In the context of the rollout, if the 'black' agent fails to achieve a checkmate within this limit, the simulation must be recorded as a loss for Black (a terminal value of 0).